Water Leakage Detection using machine learning

P A 397C
Choi, Woo Jin
(EID wc23859)
Dec 10, 2025

Contents

1. Purpose of Project
2. Dataset for Machine Learning
3. Machine Learning
4. Results and Analysis
5. Conclusion and Limitation

1. Purpose of Project

- Leakages have **got worse as pipes have aged**
  - U.S. average leakage 14% (source : U.S. EPA)
  - Worst Cases more than 60% (source : U.S. EPA)

The U.S. loses

the equivalent of over 50 million Olympic sized swimming pools of water

every year

Source: ASCE 2025 Reporting Card

- Leakage Detection still broadly relies on **manual acoustic inspection**
  - It takes unmanageable labor and time

Leakage detection by sound rode

Source : the Times

- Most **early stage** of leakage **cannot be recognized for days or even longer**
  - Despite of real-time flow and Pressure monitoring

1. Purpose of Project

- Leakages are usually **disclosed after damaging** the ground structure
  - Damage & repair cost gets bigger due to late detection
- **AI-based leakage detection** is essential for **prompt recognition**
  - Studies are in progress, but still the ultimate goal remains distant
- This project pursue leakage detection using flow and pressure data
  - In a shorter time, without on-site inspection
  - Based on small network and machine learning method learned in class
  - Performance comparison between machine learning method

2. Dataset for Machine Learning

- Operation **Date in Real Network** are mostly **confidential**
  - Even few available open dataset are **insufficient due to limited number of leakage events**
- Simulation data were generated by **Hydraulic Modeling**
  - EPANET (U.S. EPA) simulated based on Law of flow dynamics (Hazen-Williams)
  - It is world-widely used program for water supply simulation

REAL DATA ANALYSIS	SIMULATION DATA ANALYSIS
- Mostly Confidential - Leakage occurs rarely (not enough for analysis)	- High Degree of Realism - Can control the number of data including leakage

2. Dataset for Machine Learning

24 variables

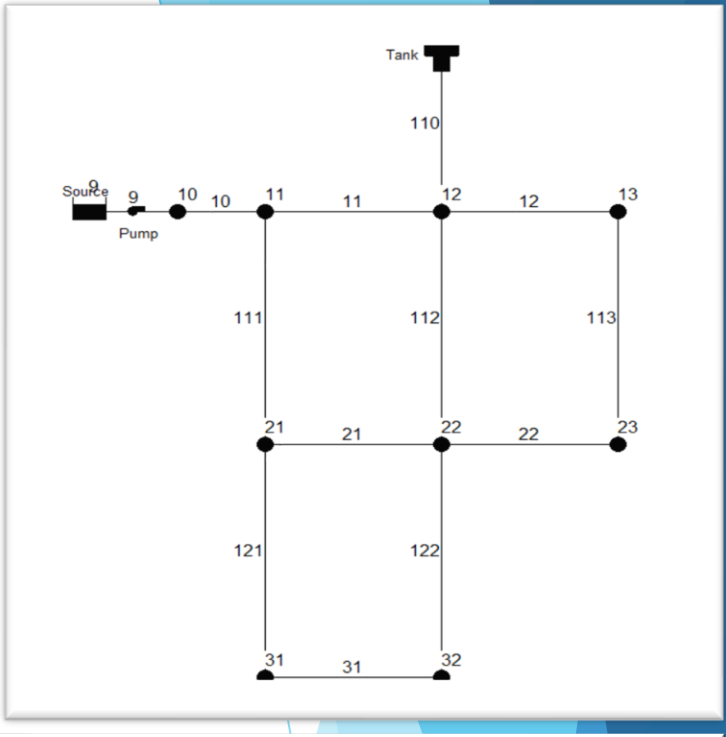
11 Node (11 pressure)

13 Link (13 flow)

9 responses

Potential Leakage at

9 Node (except WTP, TANK)



Duration

24 Hour/day, 180 days(sets), total 4,500 simul.

= (24+1) * 180

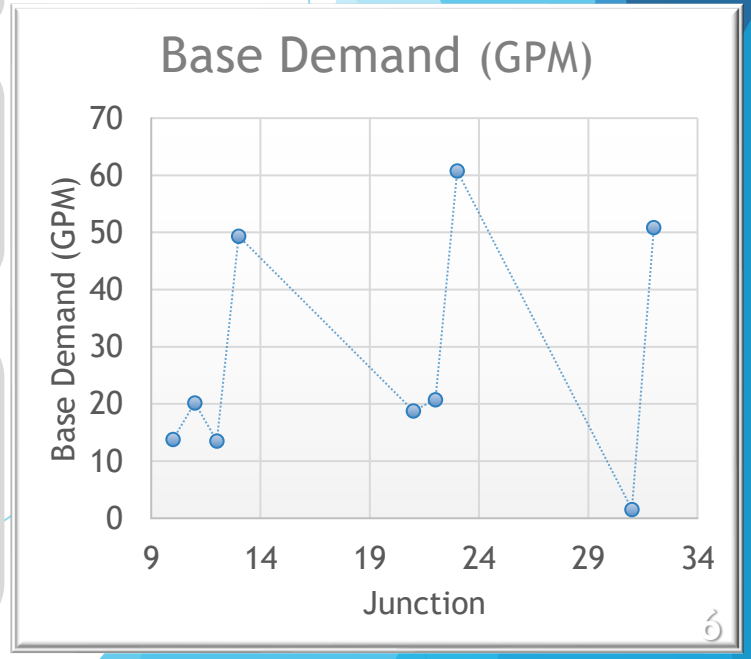
Base Demand

- 0 ~ 1000 people for each Node (random)

  (1 people = 106 Gallon Per Day (400 Liter/day))

Pattern

- Repetitive 24-hour pattern
- Randomly generated between 0~300%
- Randomly ±20% Deviation for each days

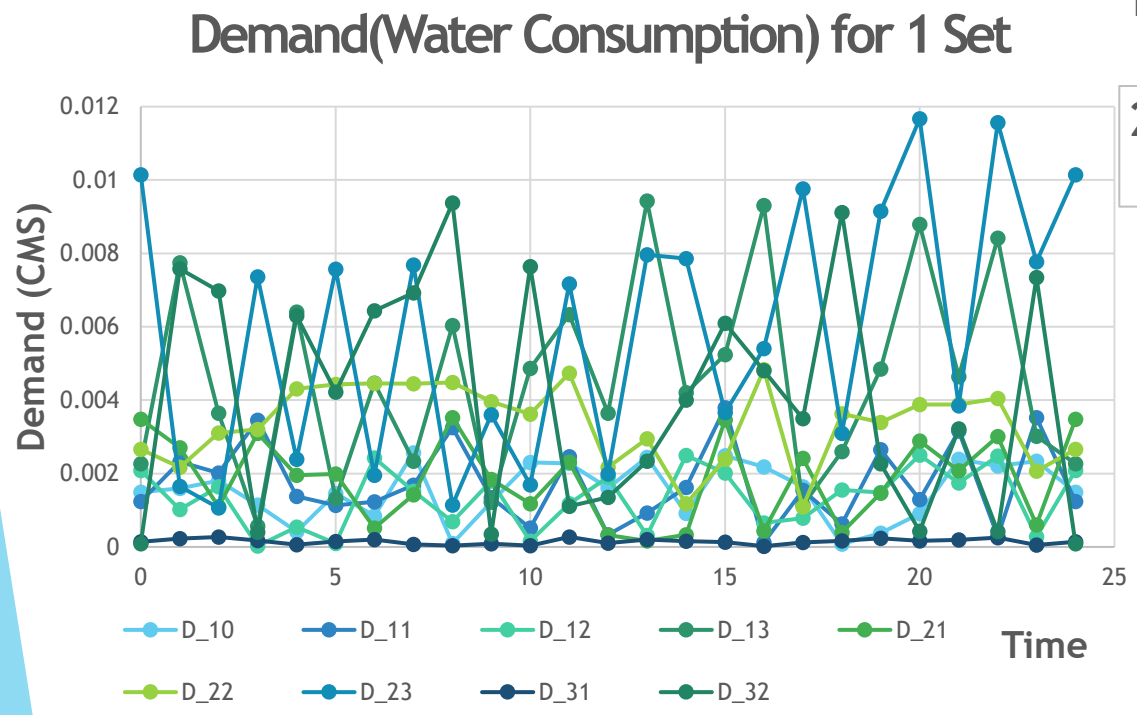


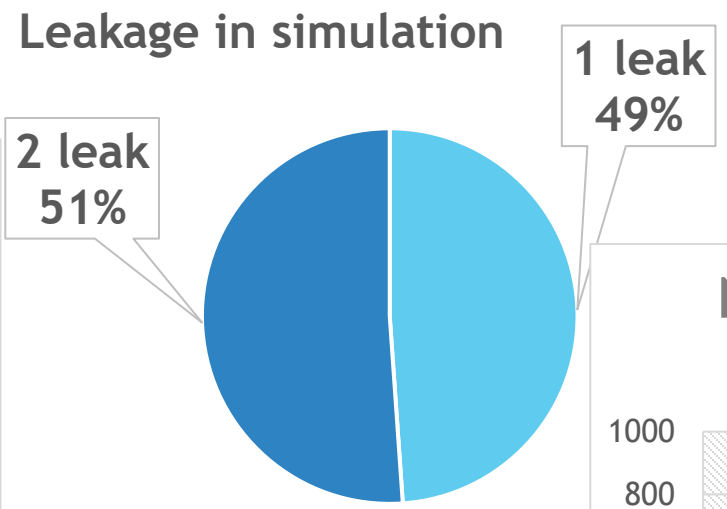
2. Dataset for Machine Learning

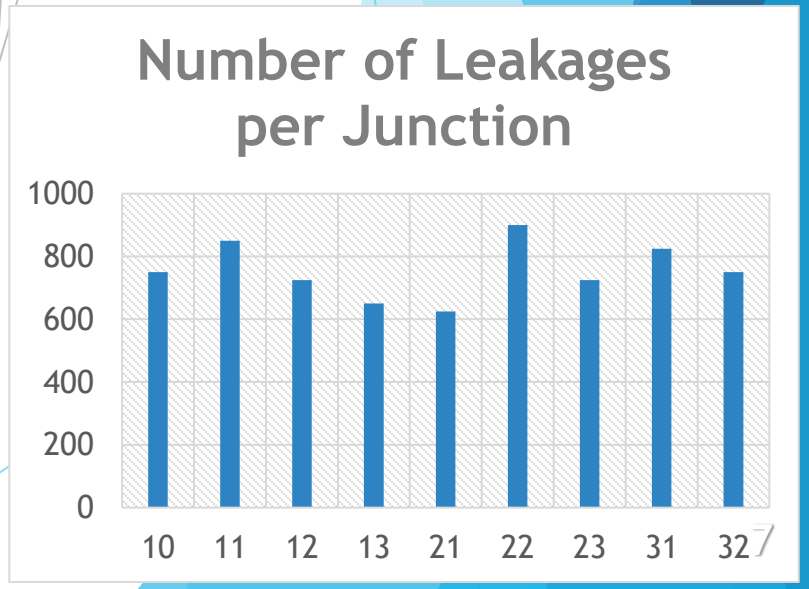
Leakage

- **1~2 Random Leakage for a Day** (1 set) ※ leakage interaction considered
- **Random amount : 0.159 ~ 15.85 GPM** (small ~ large leakage)

  - apply to EPANET with emitter coefficient (K = Q / P^0.5, Orifice Equation)







※ Demand(Water consumption) = base demand x pattern

3. Machine learning

- **Approach : 1st Stage** = General fitting for classification models

  2nd Stage = Fine Tunning for narrowed models

- **Pre-Process : ① Scaling, ② PCA** applied for only necessary Cases

Possible Model	Model Type	SCALING	PCA
Logistic Regression	Classifier	o	-
KNN	Classifier	o	-
Decision Tree	Tree	-	-
Bagging	Tree	-	-
Random Forest	Tree	-	-
Boosting	Tree	-	-
SVC Linear	Linear SVM	o	-
SVC RBF	Kernel SVM	o	o
Neural Network	Neural Network	o	-

- **Unsupervised methods were Excluded** because it is not clear for scoring
- **PCA** applied for only **time saving** (Proposition of Variance Explained : 85%), because it was not helpful for the prediction accuracy

4. Result and Analysis (1st Stage)

➢ **Through Single Split Evaluation** (70% for training, 30% for test)

Model	Recall	Precision	F1 Score	ROC_AUC
Logistic Regression	0.570	0.299	**0.364**	0.640
KNN_Classifier	0.309	0.637	**0.409**	0.737
DecisionTree	0.500	0.509	**0.504**	0.703
Bagging	0.365	0.746	**0.478**	0.803
RandomForest	0.265	0.813	**0.379**	0.809
Boosting	0.441	0.822	**0.561**	0.834
SVC_Linear	0.074	0.188	**0.093**	0.658
SVC_RBF	0.277	0.137	**0.175**	0.465
Neural Network	0.000	0.000	**0.000**	0.502

- Average of predictions accuracy for each node
- Model fit with default or minimum setting for 1st stage (refer to appendix)

❖ **Performance Criteria : F1-Score**

- Both Recall and Precision are important for leakage detection

4. Result and Analysis (1st Stage)

Distance-Based

KNN,

SVC-Radial Kernel

High Dimension of Input & Output

(24 variables, 9 response)

Linear Model

Logistic Regression

SVC-linear kernel

Non-Linear Behavior

(Flow vs Pressure)

Probability Based

Tree model

Neural Network

Better Performance

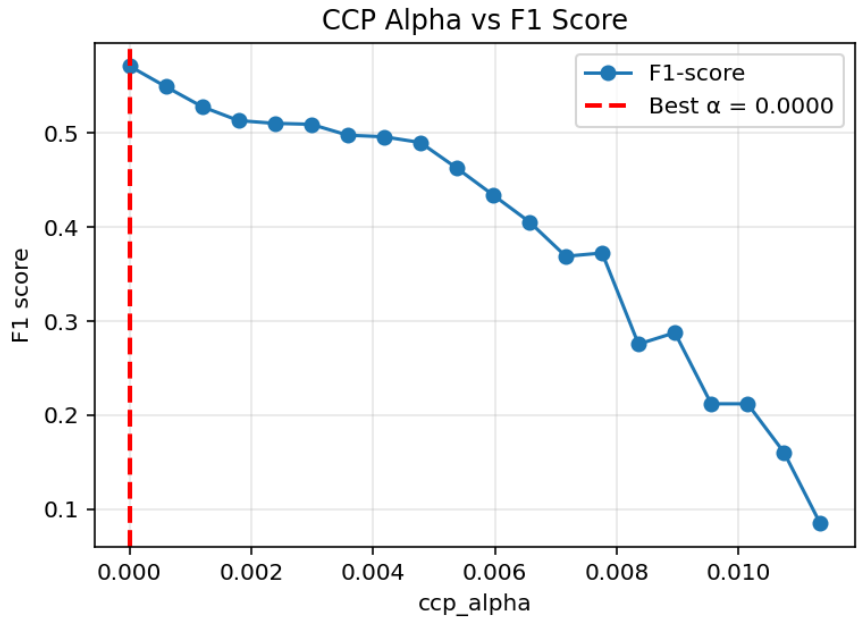
- **Overall, Tree model** performed better **within shorter time**

  - **NN** : requires much longer computation time depending on Setting (often > 30 min)

4. Result and Analysis (2nd Stage)

➢ Decision Tree

- Cost Complexity Pruning → CV(5 Fold)



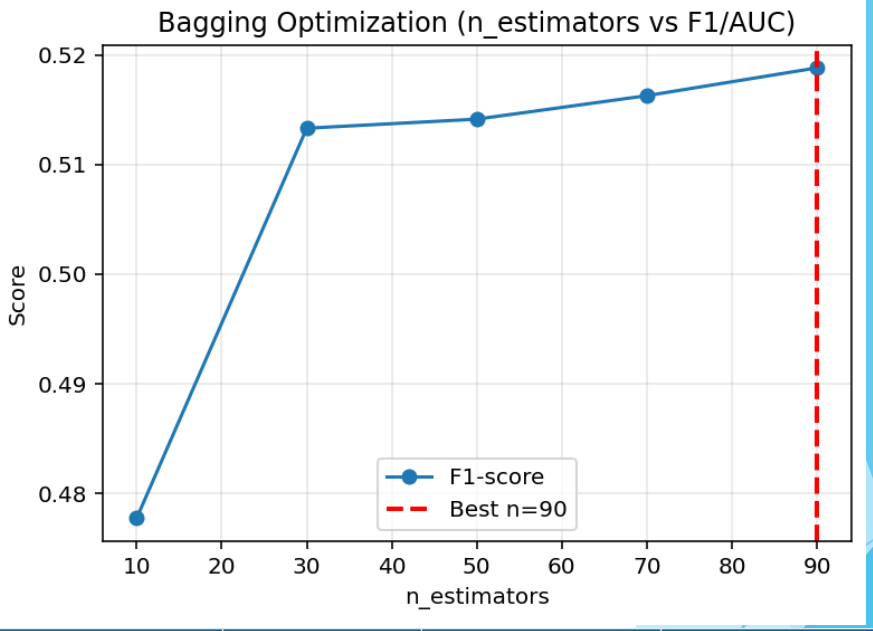
Model	Recall	Precision	F1	ROC/AUC
DecisionTree	0.500	0.509	0.504	0.703
pruned	0.493	0.502	**0.497**	0.699

✓ Pruning cannot improve performance due to complexity

✓ Tree Depth 25~42 (9 trees for each response)

➢ Bagging

- Number of Trees for bootstrap



n_estimators	Recall	Precision	F1	AUC
Default(10)	0.365	0.746	0.478	0.803
Optimized(90)	0.405	0.820	**0.519**	0.695

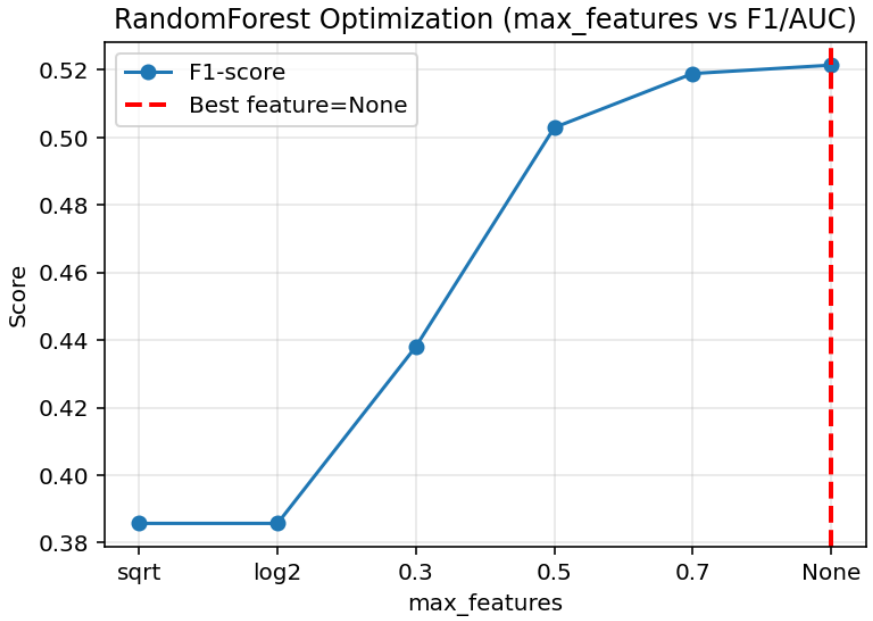
✓ The more the better, but over 30 is enough

※ Number of trees = n_estimators

4. Result and Analysis (2nd Stage)

➢ Random Forest

- Number of Features for sampling

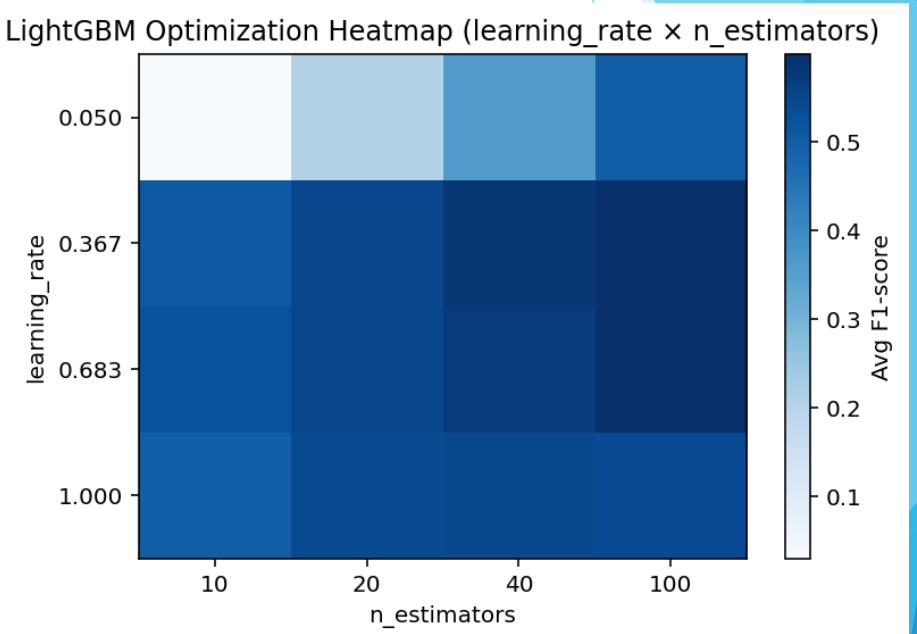


max_features	Recall	Precision	F1	AUC
Before(sqrt)	0.265	0.813	0.379	0.809
After(None)	0.404	0.824	**0.521**	0.695

✓ Feature sampling could not improve performance due to the model's complexity

➢ Boosting

- Shrinkage parameter and Number of Trees



learning_rate	estimators	Recall	Precision	F1	AUC
0.100	100	0.441	0.822	0.561	0.834
0.367	100	0.491	0.788	**0.599**	0.732

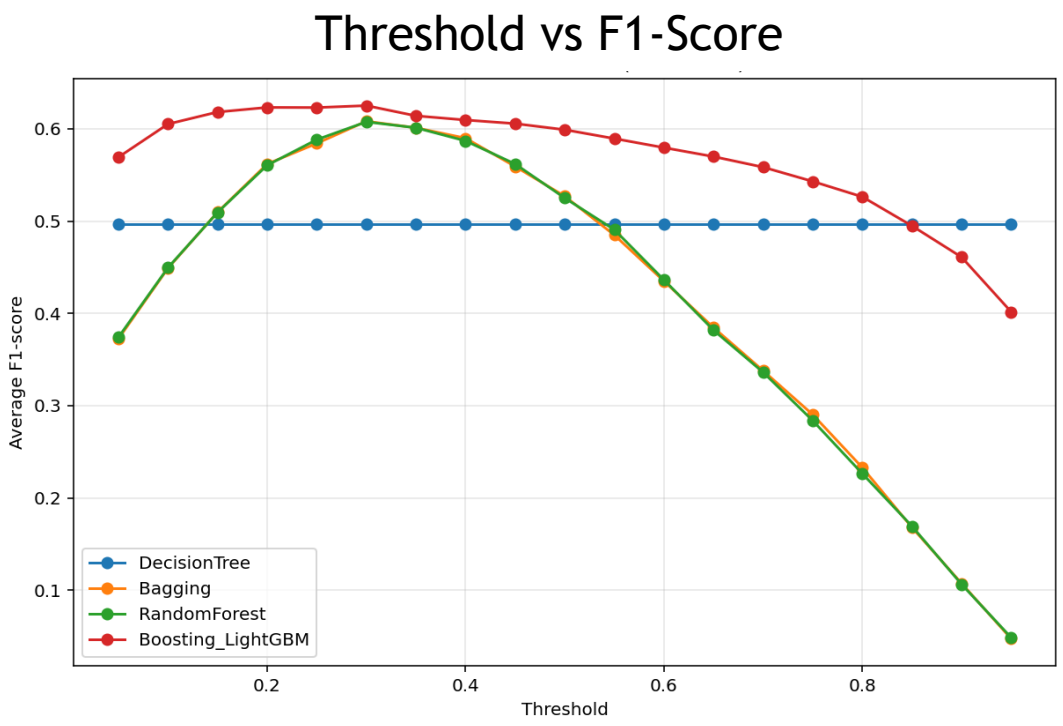
✓ Best combination is B=100, learning rate 0.367

※ Shrinkage parameter = Learning rate

4. Result and Analysis (2nd Stage)

➢ Probability Threshold Adjustment

- Recall(True Pos. Rate) is as important as Precision(Pos. Pred. Rate)
- Adjust probability thresholds for best F1-Score



Model	Rec_B	Rec_A	Pre_B	Pre_A	F1_B	F1_A	TH_B	TH_A
Decision Tree	0.493	0.493	0.502	0.502	0.497	0.497	0.500	0.050
Bagging	0.405	0.606	0.820	0.615	0.519	**0.608**	0.500	**0.300**
Random Forest	0.404	0.603	0.824	0.617	0.521	**0.608**	0.500	**0.300**
Boosting	0.491	0.567	0.788	0.701	0.599	**0.625**	0.500	**0.300**

Rec_B = Recall Before, Rec A = Recall After,
Pre_B = Precision Before, Pre_A = Precision After,
F1_B = F1 Score Before, F1_A = Fa Score After,
TH_B = Threshold Before, TH_A = Threshold After

✓ Decision tree shows highly bi-polar prediction

5. Conclusion and Limitation/Uncertainty

- **Final Score :** Best F1-score **0.625** at Boosting Model
- **Limitation/Uncertainly**

① **Small Scale** : Large Scale requires Higher-Dimension Input/Output

② **Simulation based** : Field data validation is recommended

③ **Accuracy**(Recall 56%, Precision 70%) **still low** for field application

④ **Neural Network** was excluded due to long computation time

- But it could perform better without time limitation (refer to appendix)

Additional info (network connectivity, gap between physical model and observation) needs to be incorporated for better performance

Appendix

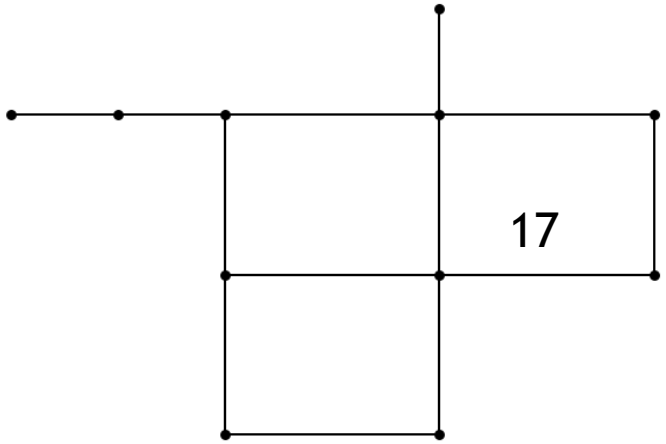
Appendix Contents

1. Simulation Data
   - Water supply network
   - Base Demand
   - Hourly Demand Pattern
   - Demand for nodes
   - Dataset for machine learning

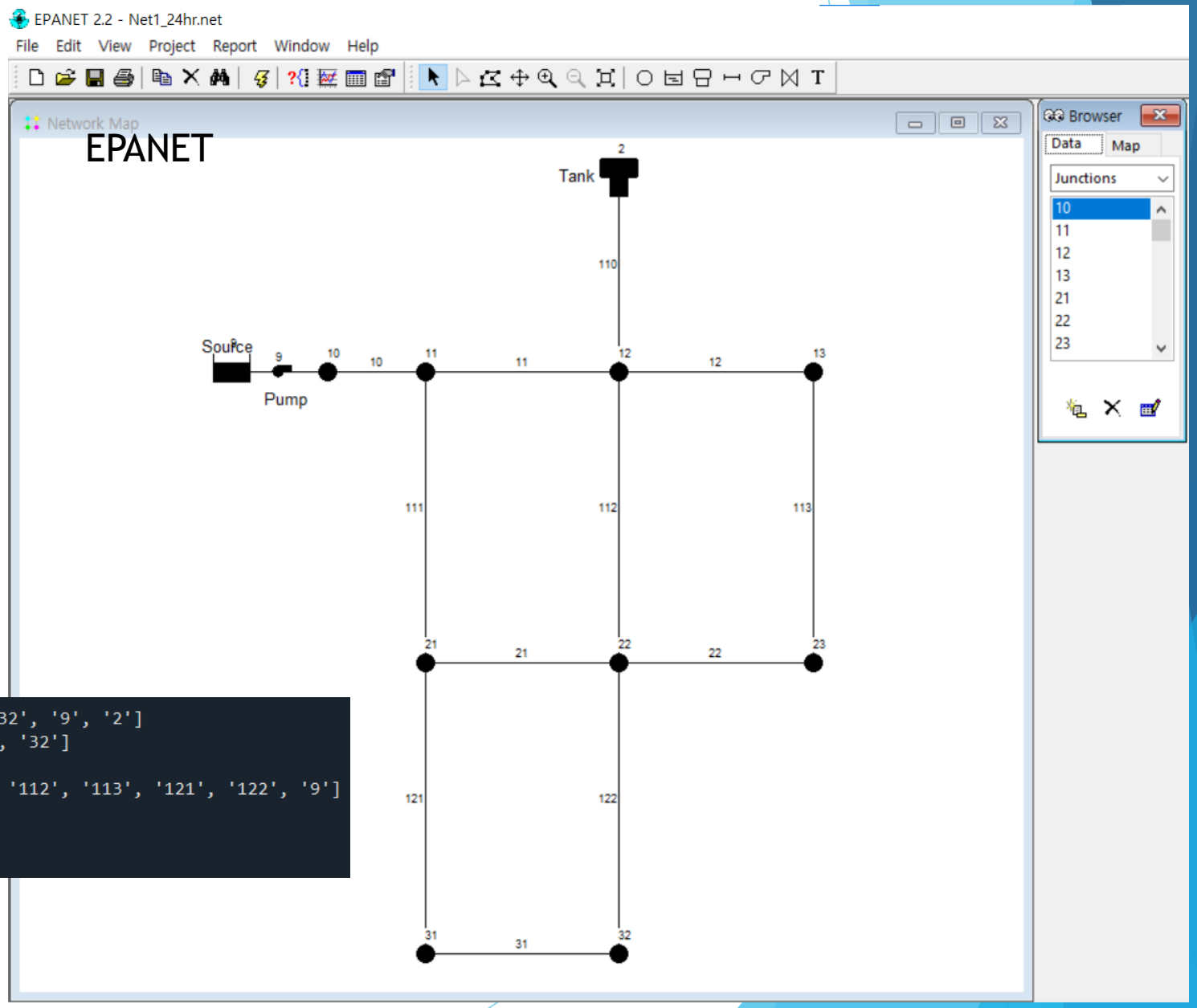
2. Machine Learning
   - Parameter Setting for Stage 1
   - Result of Stage 1
   - Result of Stage 2
   - Feature Importance
   - Additional Sensitivity Analysis
   - Neural Network model

1. Simulation Data - Water Supply Network

Python Description about network

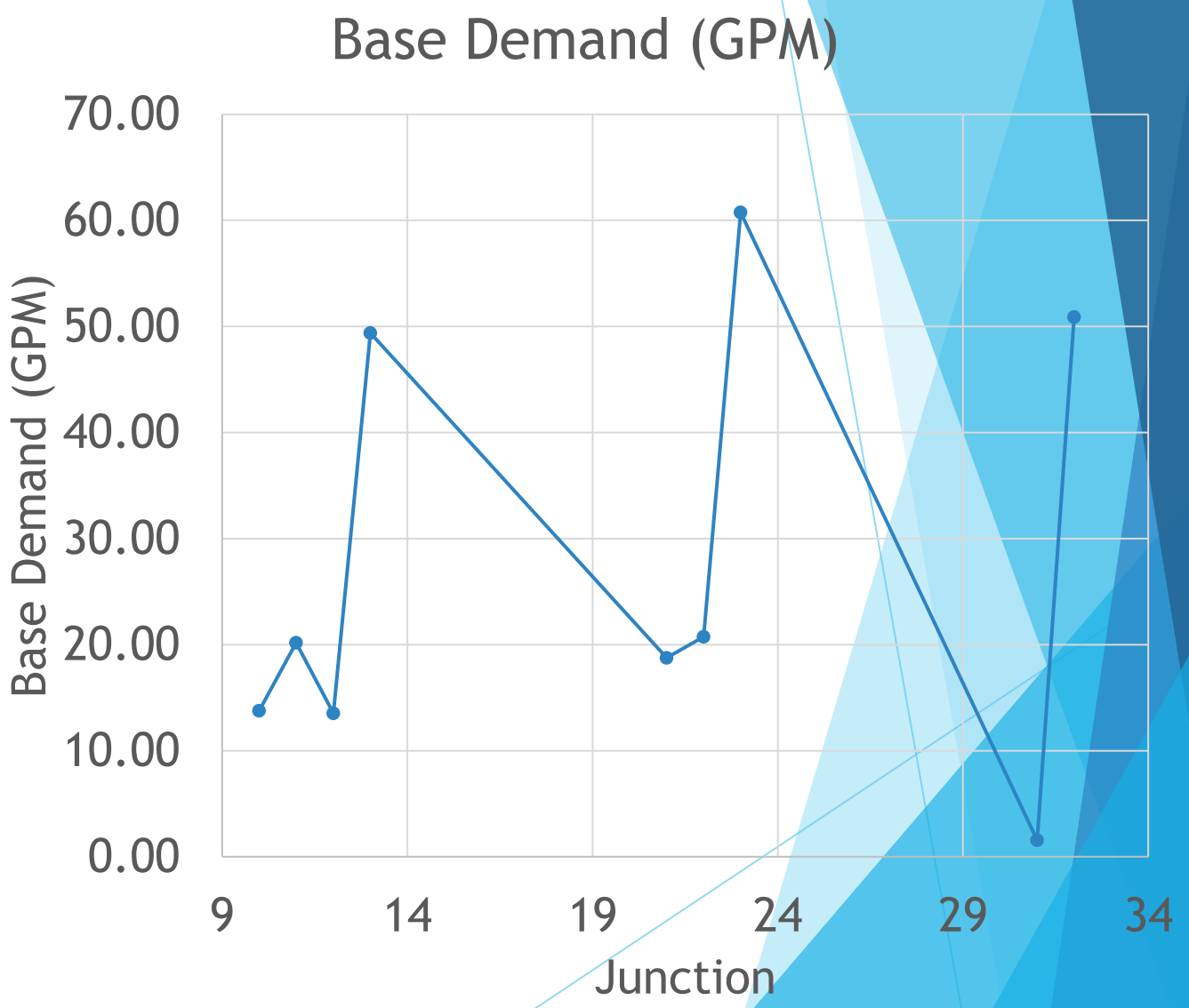



```
- node_name_list : ['10', '11', '12', '13', '21', '22', '23', '31', '32', '9', '2']
- juntion_name_list : ['10', '11', '12', '13', '21', '22', '23', '31', '32']
- total juntion number : 9
- link_name_list : ['10', '11', '12', '21', '22', '31', '110', '111', '112', '113', '121', '122', '9']
- pump_name_list : ['9']
- tank_name_list : ['2']
- demand pattern list : ['1']
```



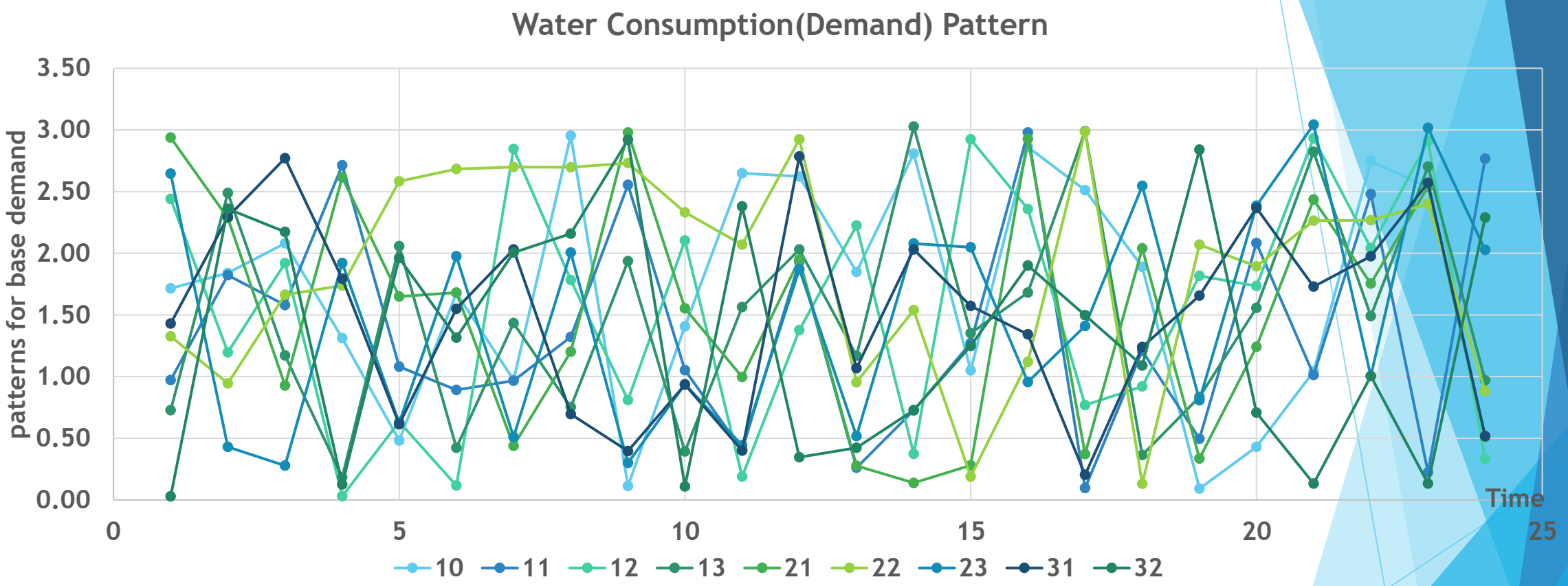
1. Simulation Data - Base Demand

Junction	Base Demand (GPM)
10	13.78
11	20.19
12	13.52
13	49.38
21	18.77
22	20.74
23	60.76
31	1.56
32	50.89



- Detailed data can be found in the separate attachment “Base_Demands.csv”

1. Simulation Data – Hourly Demand Pattern



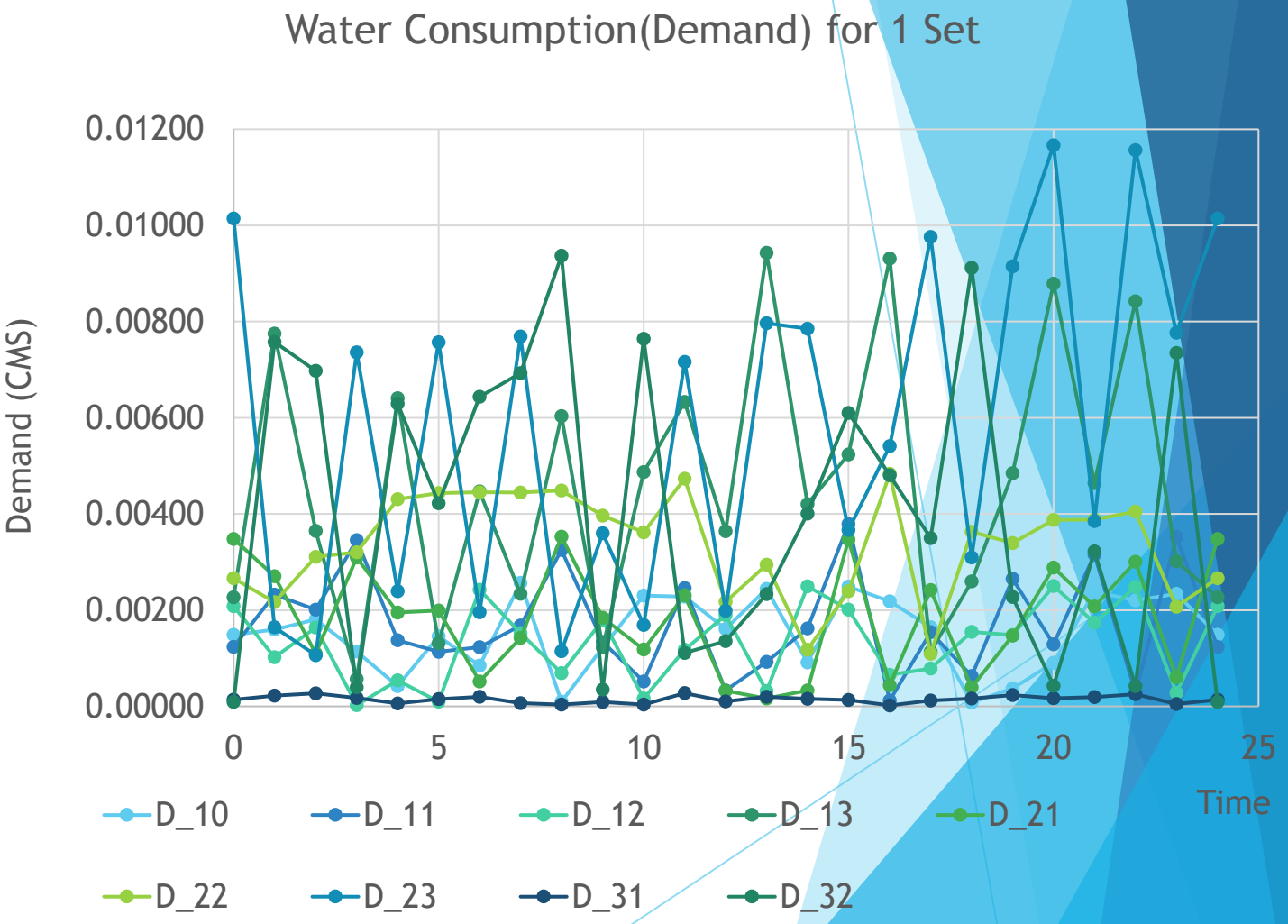
	Hour 1	Hour 2	Hour 3	Hour 4	Hour 5	Hour 6	Hour 7	Hour 8	Hour 9	Hour 10	Hour 11	Hour 12	Hour 13	Hour 14	Hour 15	Hour 16	Hour 17	Hour 18	Hour 19	Hour 20	Hour 21	Hour 22	Hour 23	Hour 24
10	1.71	1.84	2.08	1.31	0.48	1.68	0.97	2.95	0.11	1.41	2.65	2.62	1.85	2.81	1.05	2.86	2.51	1.89	0.09	0.43	1.04	2.75	2.54	2.69
11	0.97	1.82	1.58	2.71	1.08	0.89	0.97	1.32	2.55	1.05	0.40	1.93	0.26	0.73	1.27	2.98	0.10	1.21	0.50	2.08	1.01	2.48	0.22	2.77
12	2.44	1.20	1.92	0.03	0.63	0.12	2.84	1.78	0.81	2.10	0.19	1.38	2.23	0.37	2.92	2.36	0.77	0.92	1.82	1.73	2.93	2.04	2.91	0.33
13	0.73	2.49	1.17	0.18	2.06	0.42	1.43	0.75	1.94	0.39	1.56	2.03	1.17	3.03	1.35	1.68	2.99	0.36	0.83	1.56	2.82	1.49	2.70	0.97
21	2.94	2.28	0.93	2.61	1.65	1.68	0.44	1.20	2.98	1.55	1.00	1.95	0.28	0.14	0.28	2.93	0.37	2.04	0.33	1.24	2.44	1.75	2.54	0.51
22	1.33	0.94	1.66	1.74	2.58	2.68	2.70	2.70	2.73	2.33	2.07	2.92	0.95	1.54	0.19	1.12	2.99	0.13	2.07	1.89	2.26	2.27	2.40	0.88
23	2.65	0.43	0.28	1.92	0.62	1.97	0.51	2.01	0.30	0.94	0.44	1.87	0.52	2.08	2.05	0.96	1.41	2.55	0.81	2.39	3.04	1.00	3.02	2.03
31	1.43	2.29	2.77	1.79	0.61	1.55	2.03	0.70	0.40	0.93	0.40	2.78	1.07	2.03	1.57	1.34	0.20	1.24	1.66	2.37	1.73	1.98	2.57	0.52
32	0.03	2.36	2.17	0.12	1.96	1.32	2.01	2.16	2.92	0.11	2.38	0.35	0.42	0.73	1.25	1.90	1.50	1.09	2.84	0.71	0.13	1.00	0.13	2.29

- Detailed data can be found in the separate attachment “Demand_Patterns.csv”

1. Simulation Data – Demand for node (1st Set)

- Demand(water consumption) = base demand x hourly demand pattern

time	D_10	D_11	D_12	D_13	D_21	D_22	D_23	D_31	D_32
0	0.00149	0.00124	0.00208	0.00227	0.00348	0.00266	0.01014	0.00014	0.00009
1	0.00160	0.00232	0.00102	0.00775	0.00271	0.00217	0.00165	0.00023	0.00758
2	0.00181	0.00201	0.00164	0.00365	0.00110	0.00311	0.00106	0.00027	0.00698
3	0.00114	0.00346	0.00003	0.00057	0.00310	0.00320	0.00736	0.00018	0.00040
4	0.00042	0.00138	0.00054	0.00641	0.00195	0.00431	0.00239	0.00006	0.00629
5	0.00146	0.00113	0.00010	0.00131	0.00199	0.00443	0.00757	0.00015	0.00422
6	0.00085	0.00123	0.00243	0.00447	0.00052	0.00445	0.00196	0.00020	0.00644
7	0.00257	0.00168	0.00152	0.00234	0.00142	0.00445	0.00769	0.00007	0.00693
8	0.00010	0.00325	0.00069	0.00603	0.00353	0.00449	0.00115	0.00004	0.00937
9	0.00122	0.00134	0.00179	0.00122	0.00184	0.00396	0.00360	0.00009	0.00035
10	0.00230	0.00052	0.00016	0.00487	0.00118	0.00362	0.00169	0.00004	0.00764
11	0.00228	0.00246	0.00117	0.00633	0.00231	0.00473	0.00717	0.00027	0.00111
12	0.00161	0.00033	0.00190	0.00364	0.00033	0.00217	0.00198	0.00010	0.00136
13	0.00244	0.00092	0.00032	0.00943	0.00016	0.00295	0.00797	0.00020	0.00234
14	0.00091	0.00162	0.00249	0.00421	0.00033	0.00118	0.00786	0.00015	0.00401
15	0.00249	0.00379	0.00201	0.00524	0.00347	0.00239	0.00366	0.00013	0.00610
16	0.00218	0.00012	0.00066	0.00931	0.00044	0.00483	0.00541	0.00002	0.00481
17	0.00164	0.00154	0.00079	0.00114	0.00242	0.00109	0.00976	0.00012	0.00350
18	0.00008	0.00063	0.00155	0.00260	0.00040	0.00363	0.00310	0.00016	0.00912
19	0.00037	0.00265	0.00148	0.00485	0.00147	0.00340	0.00915	0.00023	0.00227
20	0.00091	0.00129	0.00250	0.00879	0.00288	0.00388	0.01167	0.00017	0.00043
21	0.00239	0.00316	0.00174	0.00464	0.00208	0.00388	0.00385	0.00019	0.00322
22	0.00220	0.00028	0.00248	0.00842	0.00301	0.00405	0.01157	0.00025	0.00042
23	0.00234	0.00352	0.00028	0.00302	0.00060	0.00207	0.00777	0.00005	0.00735
24	0.00149	0.00124	0.00208	0.00227	0.00348	0.00266	0.01014	0.00014	0.00009



- Detailed data can be found in the separate attachment “Demand.csv”

1. Simulation Data - Dataset for machine learning

Hour	N_10	N_11	N_12	N_13	N_21	N_22	N_23	N_31	N_32	N_9	N_2	F
0	87.2187	81.8713	82.4685	83.9820	82.6420	83.9923	85.5135	81.6785	78.7673	0.0000	36.5760	
1	80.4634	80.5131	83.6841	84.7647	83.0463	84.6142	86.1049	82.3580	79.5407	0.0000	38.0201	
2	79.6704	79.6708	82.7557	84.1074	82.2433	83.8638	85.3865	81.9593	78.9235	0.0000	37.0586	
3	79.0761	79.0885	82.1700	83.5286	81.9994	83.5403	85.0567	81.9135	78.8606	0.0000	36.4658	
4	78.5585	78.5630	81.6699	82.9199	81.1883	82.8333	84.3571	80.9460	78.0182	0.0000	35.9808	
5	77.8773	77.8791	80.9724	82.1200	80.4218	82.0031	83.5103	79.8402	77.0191	0.0000	35.2829	
6	77.2226	77.2278	80.2968	81.7004	80.1858	81.7102	83.2269	80.0056	76.9665	0.0000	34.5898	
7	76.6498	76.6649	79.8242	80.9754	78.9999	80.6812	82.2034	78.1967	75.5002	0.0000	34.1519	
8	85.1198	79.4433	79.8584	81.3100	80.0482	81.3767	82.8234	79.8038	76.6897	0.0000	33.9782	
9	86.2911	80.9247	81.5286	82.9793	81.5271	82.9812	84.4588	81.2222	78.1720	0.0000	35.6740	
10	87.8209	82.5271	83.0943	84.4891	83.3298	84.6175	86.1132	82.3429	79.5912	0.0000	37.2177	

Pressure at 11 nodes
(part of data)

Hour	P_10	P_11	P_12	P_21	P_22	P_31	P_110	P_111	P_112	P_113	P_121	P_122	P_9
0	0.1156	0.0934	0.0016	0.0082	0.0012	-0.0025	-0.0912	0.0205	0.0002	-0.0005	0.0079	0.0072	0.1214
1	-0.0092	-0.0186	0.0123	-0.0039	0.0045	-0.0033	0.0497	0.0092	0.0147	0.0031	0.0066	0.0062	0.0000
2	-0.0007	-0.0097	0.0074	-0.0060	0.0008	-0.0007	0.0306	0.0088	0.0121	0.0036	0.0041	0.0053	0.0000
3	-0.0044	-0.0092	0.0072	-0.0023	0.0020	0.0004	0.0251	0.0045	0.0071	-0.0004	0.0021	0.0027	0.0000
4	-0.0025	-0.0125	0.0095	-0.0067	0.0003	-0.0023	0.0361	0.0082	0.0112	0.0020	0.0037	0.0042	0.0000
5	-0.0015	-0.0108	0.0113	-0.0045	0.0031	-0.0033	0.0358	0.0091	0.0133	0.0026	0.0060	0.0056	0.0000
6	-0.0027	-0.0071	0.0061	-0.0003	0.0020	-0.0006	0.0226	0.0036	0.0059	-0.0003	0.0032	0.0035	0.0000
7	-0.0049	-0.0176	0.0112	-0.0078	0.0009	-0.0041	0.0456	0.0109	0.0157	0.0040	0.0072	0.0069	0.0000
8	0.1194	0.0971	0.0046	0.0087	0.0071	0.0017	-0.0877	0.0213	0.0012	0.0007	0.0038	0.0028	0.1244
9	0.1158	0.0932	0.0047	0.0050	0.0054	0.0003	-0.0798	0.0213	0.0047	0.0014	0.0042	0.0041	0.1228
10	0.1150	0.0940	0.0063	0.0097	0.0041	-0.0038	-0.0866	0.0203	0.0004	-0.0022	0.0080	0.0059	0.1205

Flow at 13 links
(part of data)

Hour	L_N_10_Leak	L_N_11_Leak	L_N_12_Leak	L_N_13_Leak	L_N_21_Leak	L_N_22_Leak	L_N_23_Leak	L_N_31_Leak	L_N_32_Leak
0	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
1	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
2	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
3	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
4	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
5	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
6	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
7	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
8	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
9	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
10	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE

Leakage at 9 nodes
(part of data)

- Detailed data can be found in the separate attachment “Combined_All.csv”

2. Machine Learning - Parameter Setting for Stage 1

```python
# Tree-based multi-output models
tree_models = {
    "DecisionTree": MultiOutputClassifier(
        DecisionTreeClassifier(class_weight="balanced")),

    "Bagging": MultiOutputClassifier(
        BaggingClassifier(
            estimator=DecisionTreeClassifier(class_weight="balanced"),
            n_jobs=-1,
            random_state=seed)),

    "RandomForest": MultiOutputClassifier(
        RandomForestClassifier(
            class_weight="balanced",
            max_features="sqrt",
            random_state=seed))
    }

# Boosting (single-output based)
boosting_models = {
    "Boosting_LightGBM": LGBMClassifier(
        objective='binary',
        random_state=seed,
        n_jobs=-1,
        verbose=-1)
    }
```

```python
    "Logistic": LogisticRegression(class_weight='balanced', max_iter=3000),

    "KNN_Classifier": KNeighborsClassifier(
        # n_neighbors=10,
        weights='distance'),

    "SVC_Linear": LinearSVC(
        C=0.3,
        class_weight='balanced',
        max_iter=10000),

    "SVC_RBF": SVC(kernel='rbf',
        C=0.3,
        max_iter=10000,
        gamma=0.04,
        class_weight='balanced'),
    }
```

```python
# NN (Multi-output)
nn_models = {
    "NN_MLP_Multi": MultiOutputClassifier(
        MLPClassifier(
            hidden_layer_sizes=(128, 64),
            solver='adam',          # lbfgs → adam
            activation='relu',
            max_iter=5000,          # 10000까지 안 가도 충분
            learning_rate_init=0.001,
            early_stopping=True,    # adam에서만 의미 있음
            random_state=seed))
    }
```

❖ Detailed code can be found in the separate attachment "Leakage_Detection_ML_Project_Woo Jin Choi_final.py"

2. Machine Learning - Result of Stage 1

```
================= PERFORMANCE SUMMARY =================

                Model  Avg_Recall  Avg_Precision    Avg_F1  Avg_ROC_AUC
0        DecisionTree    0.499844       0.509213  0.503593     0.703007
1             Bagging    0.365242       0.745972  0.477684     0.802950
2        RandomForest    0.265019       0.813495  0.378919     0.808755
3        NN_MLP_Multi    0.709948       0.721602  0.715174     0.900465
4            Logistic    0.569617       0.299128  0.363526     0.640216
5      KNN_Classifier    0.308745       0.636541  0.409462     0.736597
6          SVC_Linear    0.073806       0.188034  0.093217     0.657547
7             SVC_RBF    0.276549       0.136675  0.174806     0.464860
8  Boosting_LightGBM    0.441112       0.821903  0.561057     0.833800

=======================================================
```

- Detailed code can be found in the separate attachment "Leakage_Detection_ML_Project_Woo Jin Choi_final.py"

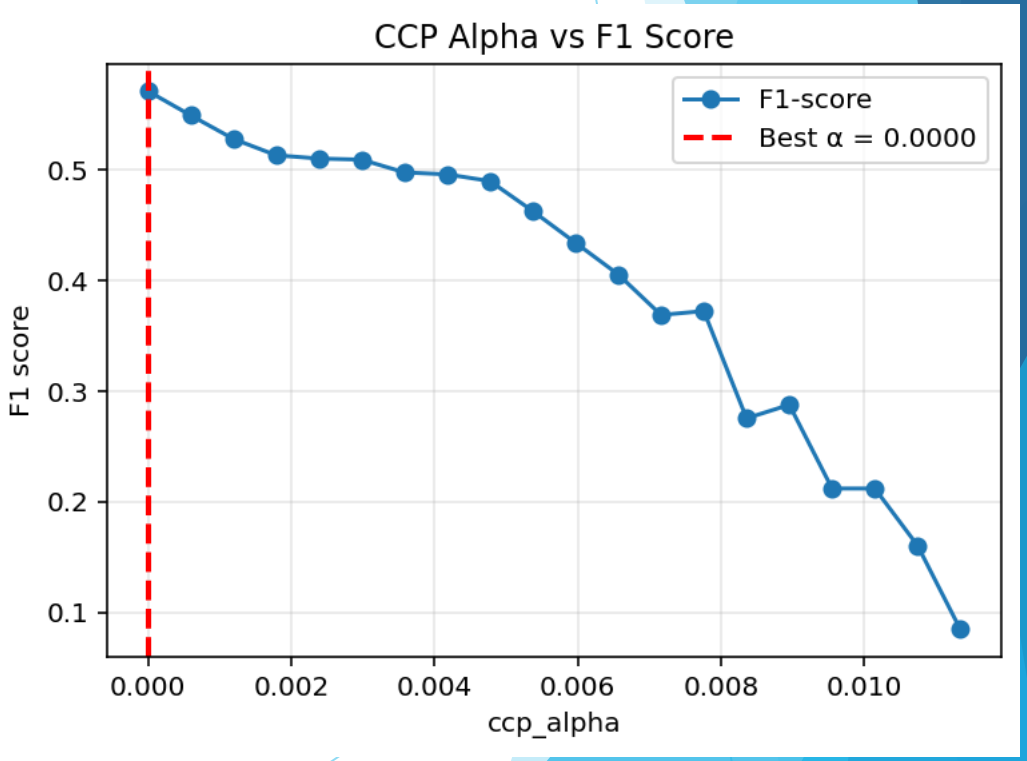
2. Machine Learning - Result of Stage 2 (Tunning)

❖ Decision Tree : result of both Cost Complexity Pruning and selection of best tree model based on Cross Validation

```
==================== DecisionTree α Optimization (CCP) ====================
▶ Best alpha = 0.0000


========= Comparison between before vs after (DecisionTree) ============
                 Model  Avg_Recall  Avg_Precision    Avg_F1  Avg_ROC_AUC
0         DecisionTree    0.499844       0.509213  0.503593     0.703007
0  DecisionTree_pruned    0.493337       0.501939  0.497112     0.698501
============================================================================
```

```
==================== DecisionTree Depth Summary ====================
Min depth : 25
Max depth : 42
Avg depth : 31.89
====================================================================
```



❖ Detailed code can be found in the separate attachment "Leakage_Detection_ML_Project_Woo Jin Choi_final.py"

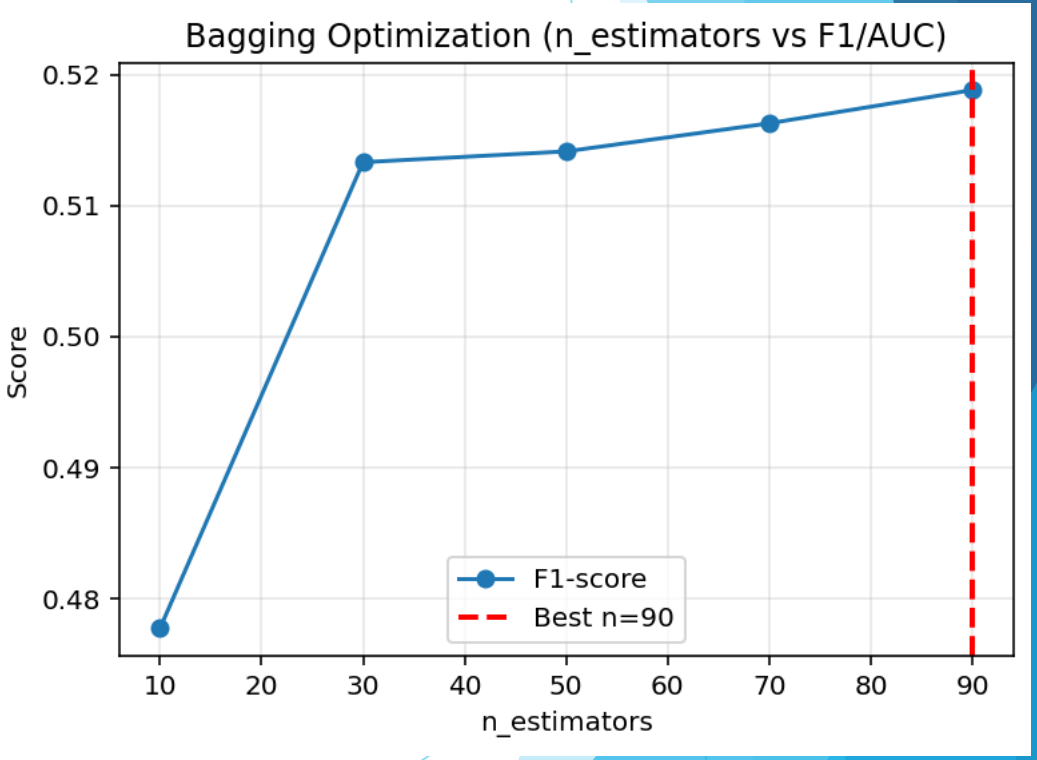
2. Machine Learning - Result of Stage 2 (Tunning)

❖ Bagging : Determining optimal number of tree for bootstrap

```
=============== Bagging n_estimators Optimization =======
[Bagging] n=10 → F1=0.4777
[Bagging] n=30 → F1=0.5133
[Bagging] n=50 → F1=0.5141
[Bagging] n=70 → F1=0.5163
[Bagging] n=90 → F1=0.5188

Best Bagging result → n=90, F1=0.5188
```

```
======= Bagging OPtimization Result ========
                    n_estimators    Recall  Precision        F1       AUC
Before(default=10)            10  0.365242   0.745972  0.477684  0.802950
After(optimized)              90  0.405294   0.820413  0.518806  0.694893
```



❖ Detailed code can be found in the separate attachment "Leakage_Detection_ML_Project_Woo Jin Choi_final.py"

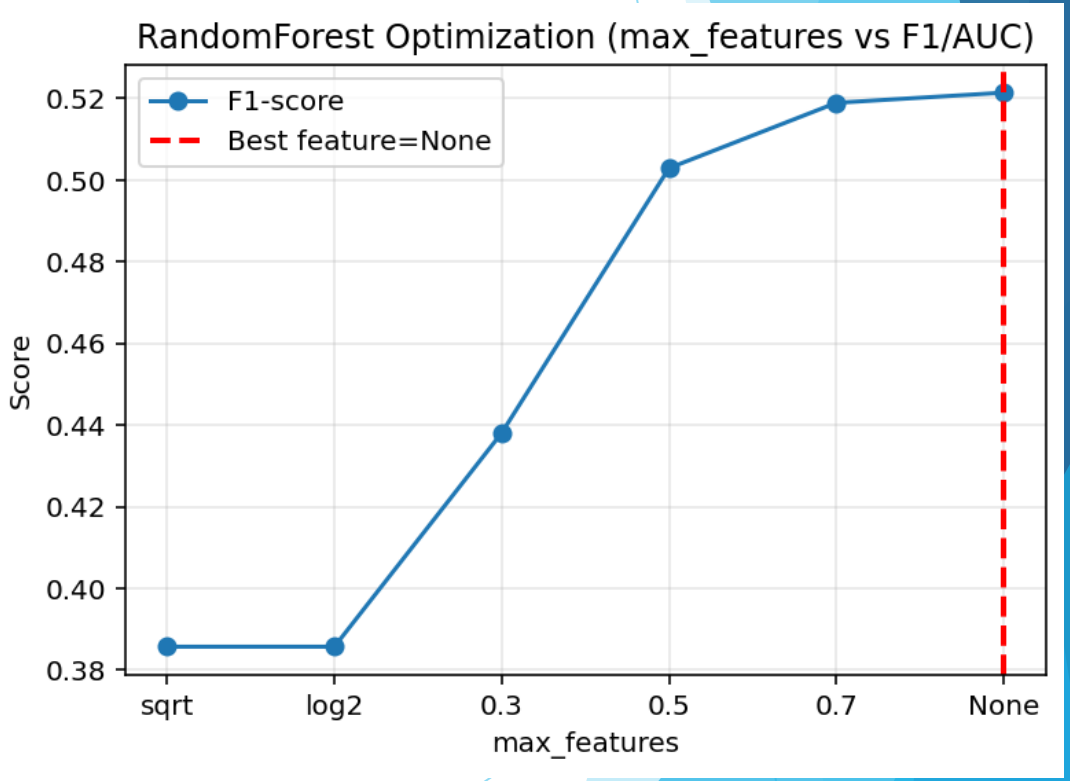
2. Machine Learning - Result of Stage 2 (Tunning)

❖ Random Forest : Determining optimal extraction of variables

```
=============== RandomForest max_features Optimization ====
max_features=sqrt → Avg_F1=0.3857
max_features=log2 → Avg_F1=0.3857
max_features=0.3 → Avg_F1=0.4381
max_features=0.5 → Avg_F1=0.5029
max_features=0.7 → Avg_F1=0.5188
max_features=None → Avg_F1=0.5213

Best RF max_feature = None | F1=0.5213
```

```
======= RandomForest Optimization Result ========
         max_features    Recall  Precision        F1       AUC
Before  sqrt(default)  0.265019   0.813495  0.378919  0.808755
After            None  0.403840   0.823509  0.521341  0.695488
```



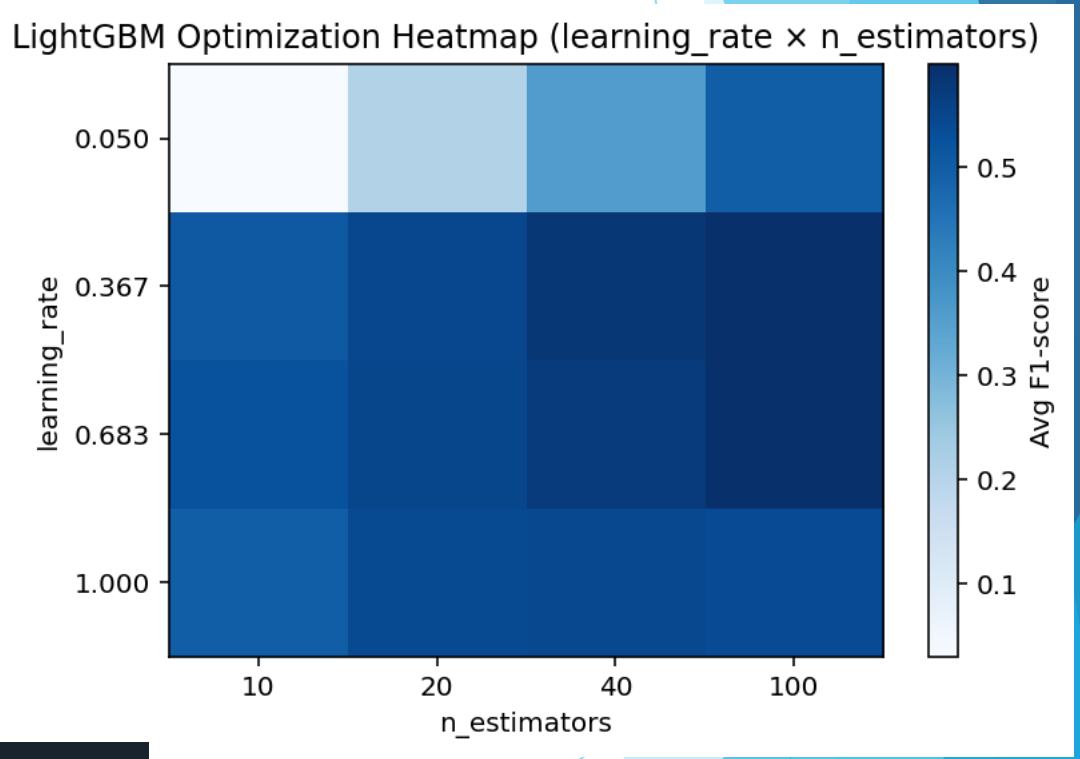
❖ Detailed code can be found in the separate attachment "Leakage_Detection_ML_Project_Woo Jin Choi_final.py"

2. Machine Learning - Result of Stage 2 (Tunning)

❖ Boosting : Determining optimal shrinkage parameter(learning rate) and number of trees

```
============ Boosting Optimization (learning_rate × n_estimators) =
lr=0.0500, n_estimators=10 → F1=0.0306
lr=0.0500, n_estimators=20 → F1=0.2064
lr=0.0500, n_estimators=40 → F1=0.3581
lr=0.0500, n_estimators=100 → F1=0.4976
lr=0.3667, n_estimators=10 → F1=0.5082
lr=0.3667, n_estimators=20 → F1=0.5468
lr=0.3667, n_estimators=40 → F1=0.5821
lr=0.3667, n_estimators=100 → F1=0.5990
lr=0.6833, n_estimators=10 → F1=0.5262
lr=0.6833, n_estimators=20 → F1=0.5493
lr=0.6833, n_estimators=40 → F1=0.5725
lr=0.6833, n_estimators=100 → F1=0.5972
lr=1.0000, n_estimators=10 → F1=0.4983
lr=1.0000, n_estimators=20 → F1=0.5400
lr=1.0000, n_estimators=40 → F1=0.5443
lr=1.0000, n_estimators=100 → F1=0.5383

>>> Best LGB Params → LR=0.367, Estimators=100, F1=0.5990
```




```
======= LightGBM Optimization Result (Unified Format) ========
        learning_rate  n_estimators    Recall  Precision        F1      AUC
Before       0.100000           100  0.441112   0.821903  0.561057  0.83380
After        0.366667           100  0.490513   0.787589  0.599012  0.73249
```

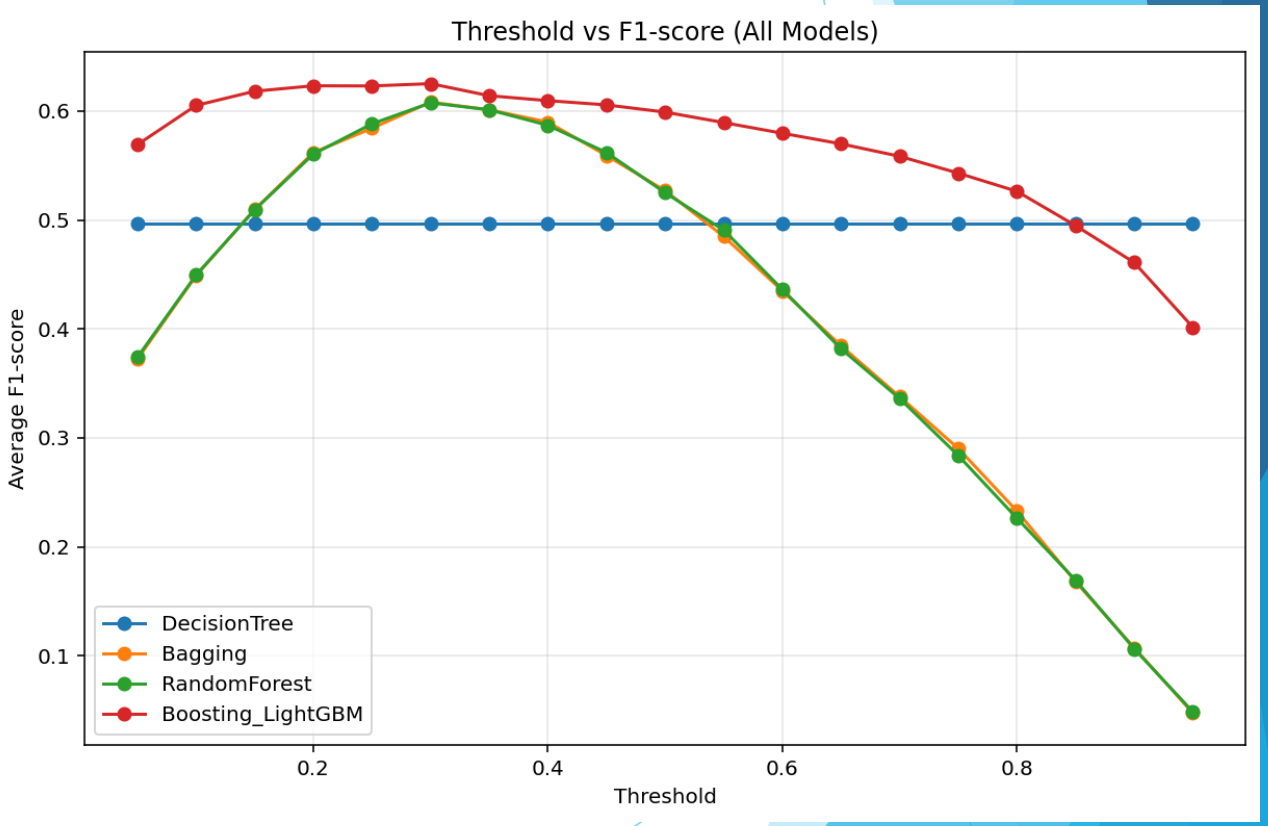
❖ Detailed code can be found in the separate attachment "Leakage_Detection_ML_Project_Woo Jin Choi_final.py"

2. Machine Learning - Result of Stage 2 (Tunning)

➢ Determining Probability Threshold for optimal F1-score

```
========== FINAL THRESHOLD OPTIMIZATION RESULT =======
             Model    Rec_B    Rec_A    Pre_B    Pre_A
      DecisionTree 0.493337 0.493337 0.501939 0.501939
           Bagging 0.405294 0.605743 0.820413 0.614604
      RandomForest 0.403840 0.602962 0.823509 0.617290
 Boosting_LightGBM 0.490513 0.566532 0.787589 0.701255
=====================================================
```

```
====
     F1_B     F1_A    AUC_B    AUC_A  TH_B  TH_A
 0.497112 0.497112 0.698501 0.698501   0.5  0.05
 0.518806 0.608175 0.694893 0.763468   0.5  0.30
 0.521341 0.607539 0.695488 0.762151   0.5  0.30
 0.599012 0.625144 0.732490 0.758428   0.5  0.30
=====
```

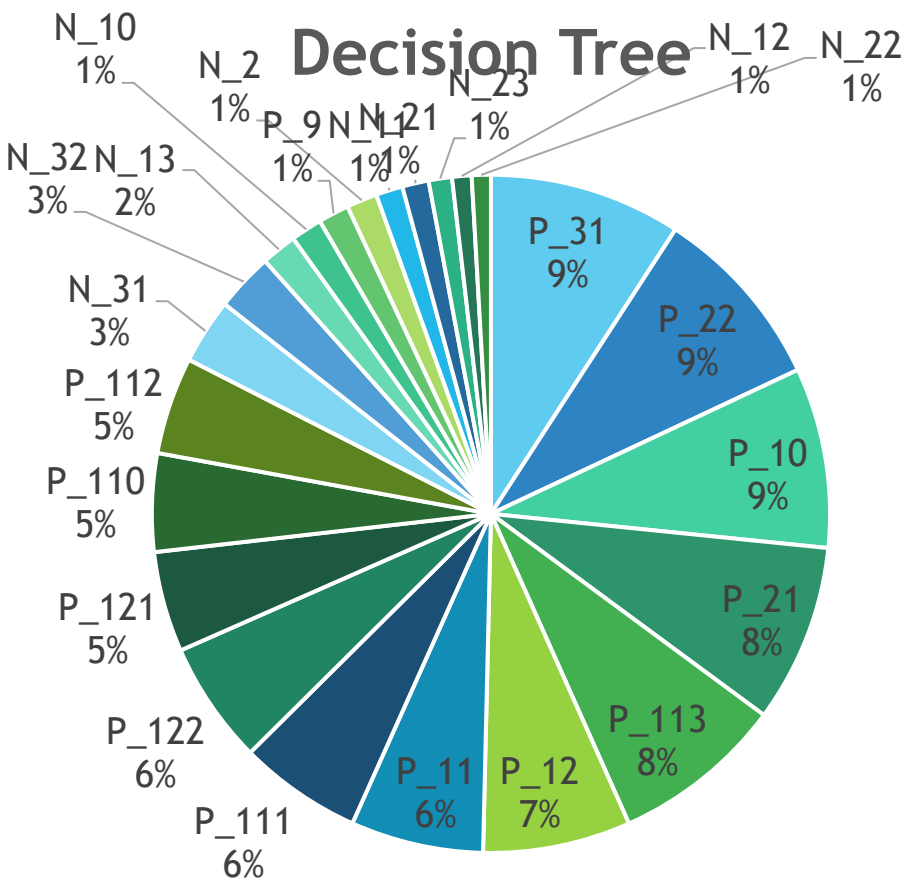


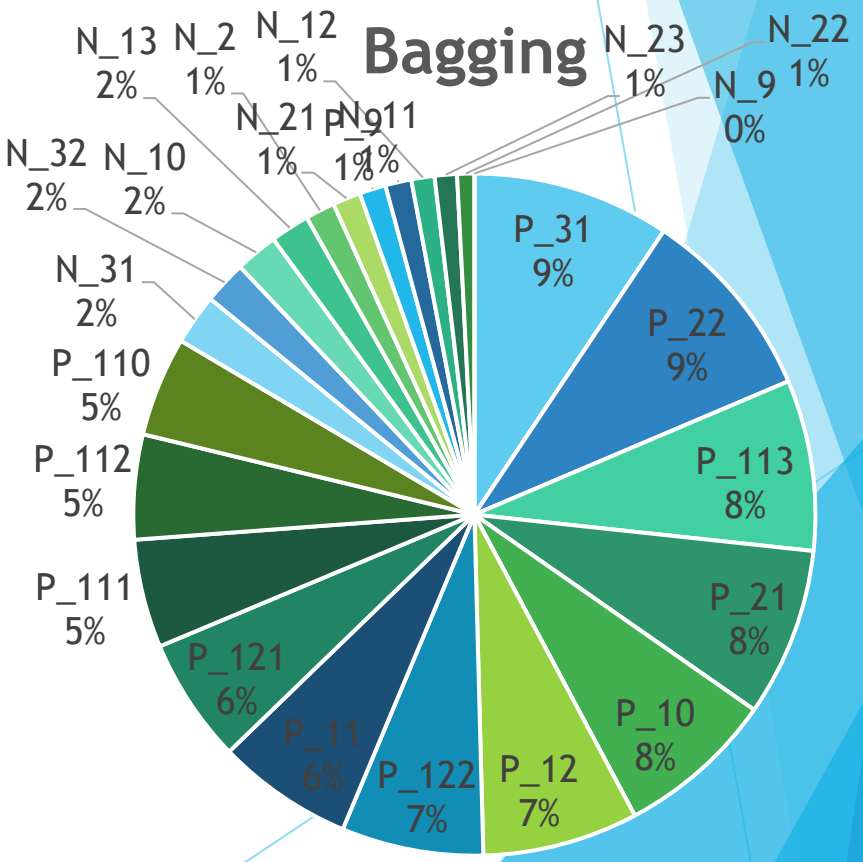
❖ Detailed code can be found in the separate attachment "Leakage_Detection_ML_Project_Woo Jin Choi_final.py"

2. Machine Learning - Feature Importance

- **Flow data at Links** shows **higher importance** than Pressure data at Nodes

→ Flow change are localized at leakage Node, while pressure changes across multiple Nodes



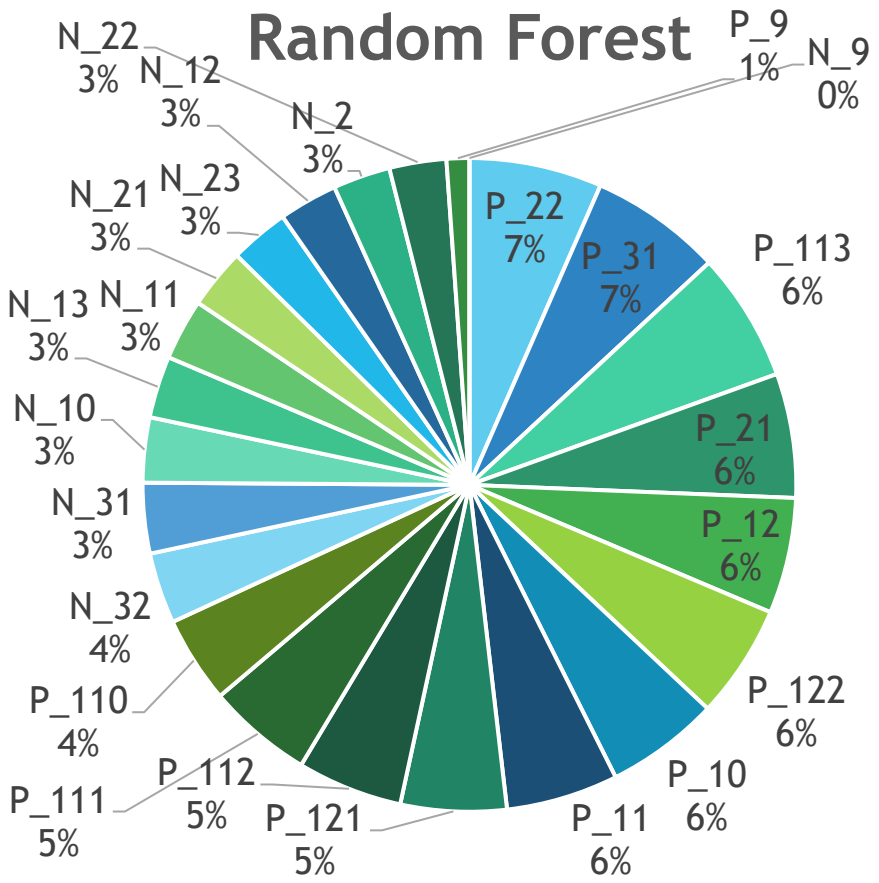


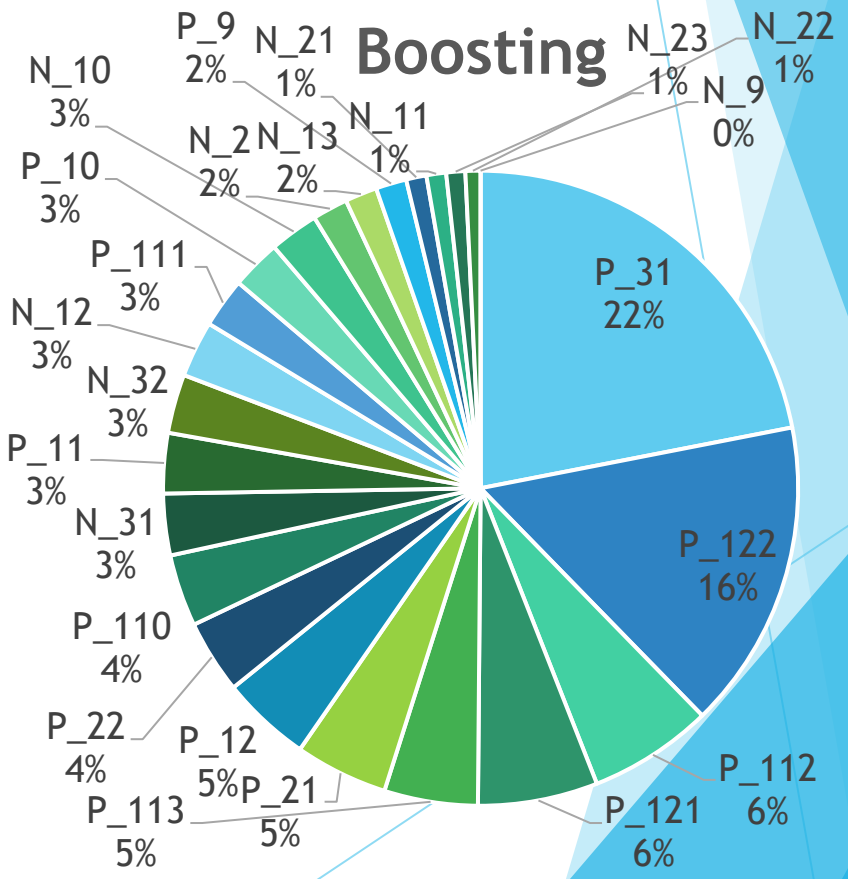
- Detailed date can be found in the separate attachment "ML_Evaluation_Results_Final.xlsx"

2. Machine Learning - Feature Importance

- Only **Boosting model** shows highly **concentrated feature importance**

→ Boosting possibly tend to focus on small number of features during learning



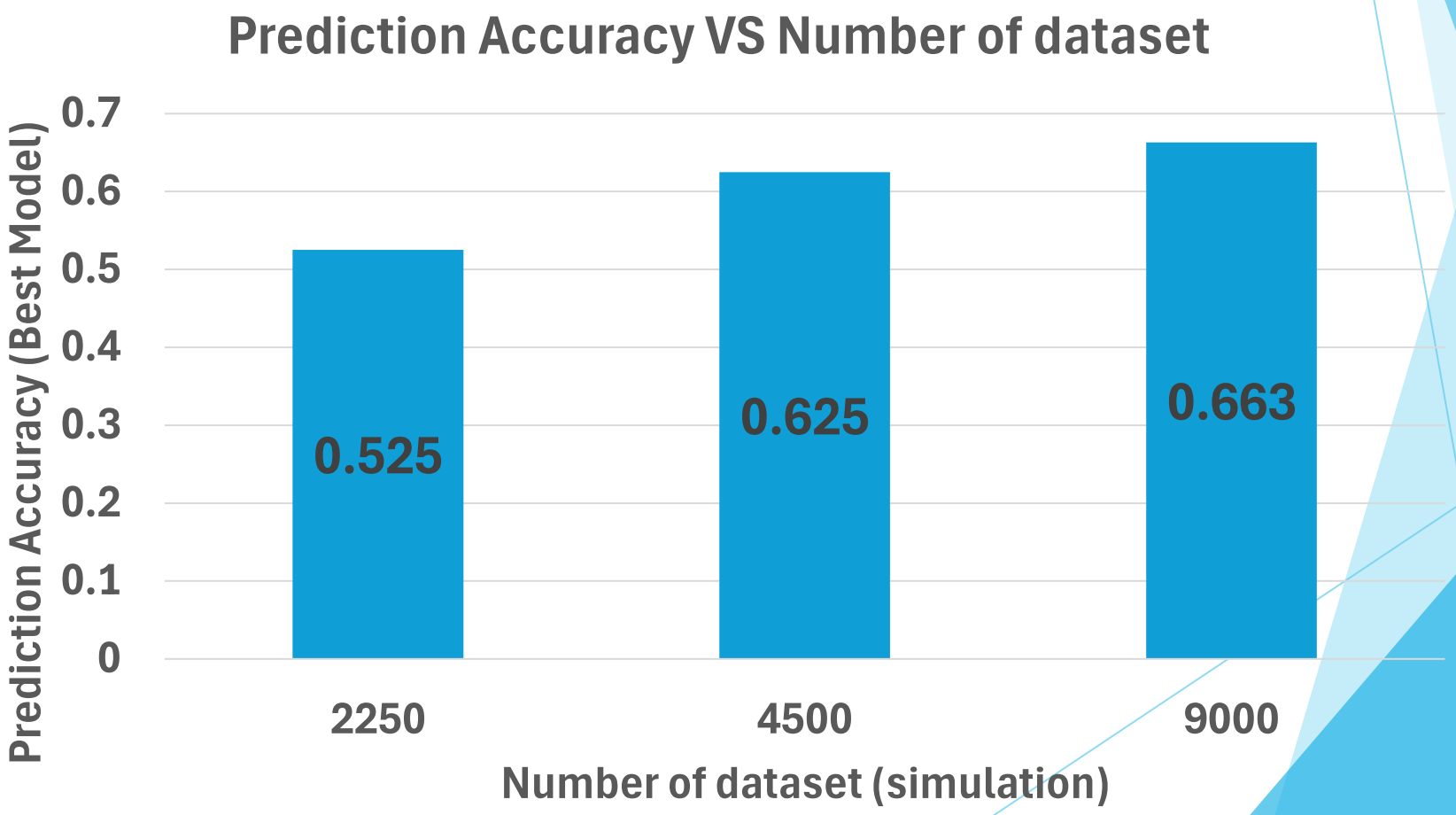


- Detailed date can be found in the separate attachment “ML_Evaluation_Results_Final.xlsx”

2. Machine Learning - Additional Sensitivity Study

- Sensibility to **dataset size**

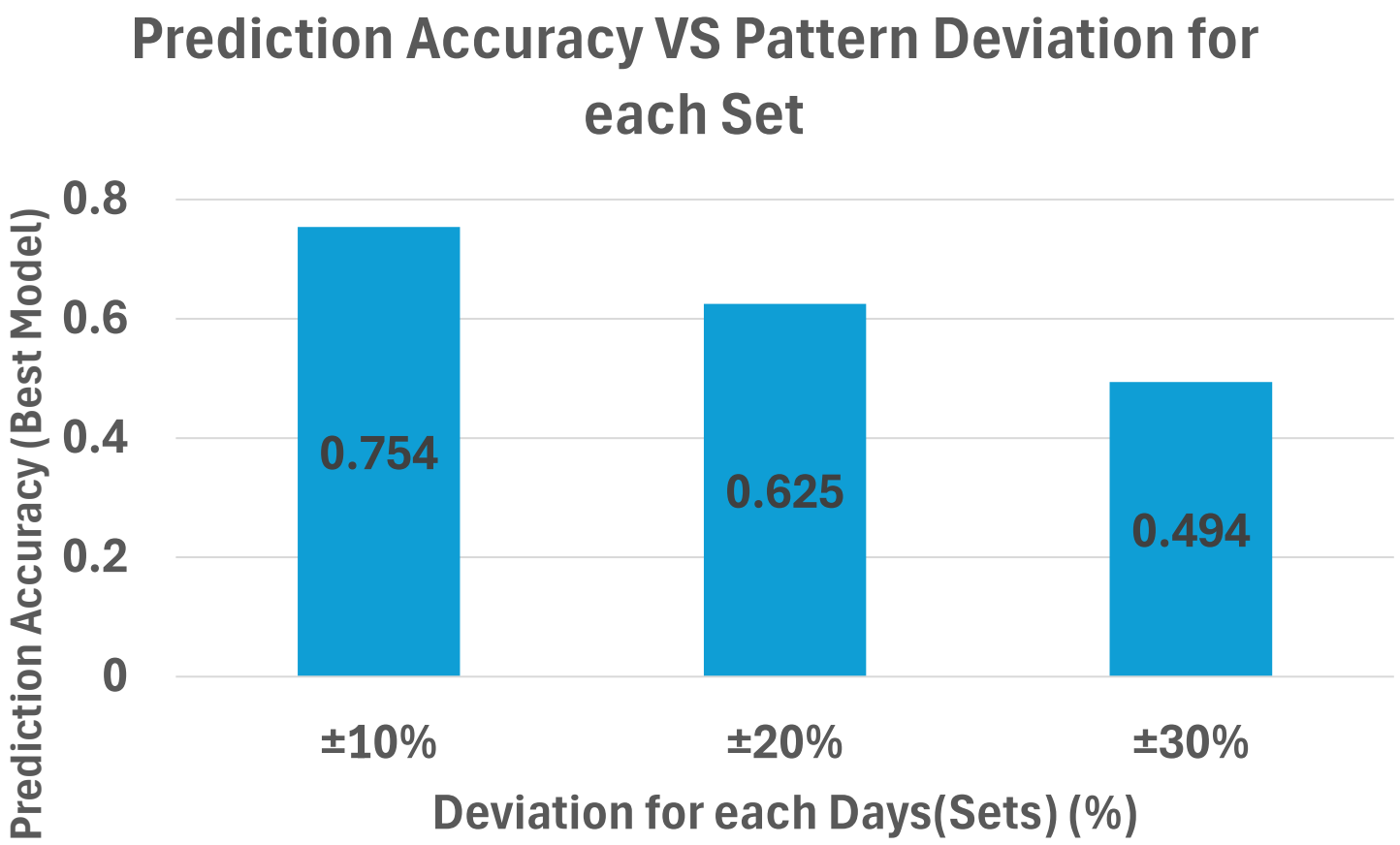
  → Larger dataset led to better performance



2. Machine Learning - Additional Sensitivity Study

- Sensibility to **Demand pattern deviation** for each set

  → Larger deviation lowered performance

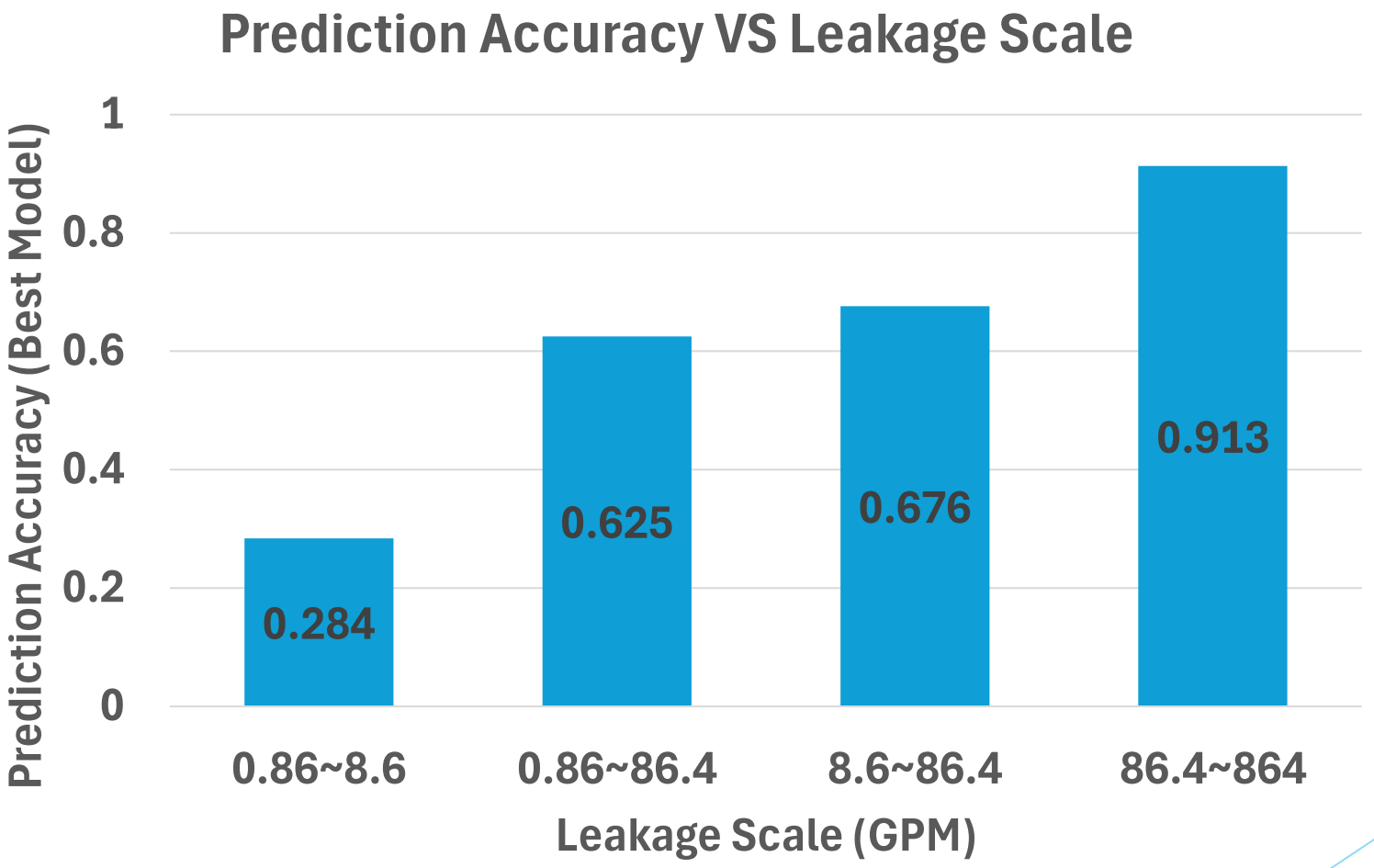


- ✓ Most daily water-consumption patterns are expected to deviate by no more than 20%
- ✓ Applying different patterns for weekend and holiday can also be considered.

2. Machine Learning - Additional Sensitivity Study

- Sensibility to **Leakage Size**

  → Larger leakages can be detected better



- ✓ Current analysis range(0.86~86.4 gpm) covers reasonable small to large leakages
- ✓ For better detection, difference in both flow and pressure can be further investigated.

2. Machine Learning - Neural Network (NN)

- With Solver ‘lbfgs’, NN model performed with **F1-Score 0.715**
  - This project **excluded** extensive **test of NN models**, as **some required over 30 min to run**, making them **unsuitable for real-time monitoring**
  - However, when real-time monitoring is not essential, NN model **can be best option** for leakage detection

```
================ PERFORMANCE SUMMARY ================
```

	Model	Avg_Recall	Avg_Precision	Avg_F1	Avg_ROC_AUC
0	DecisionTree	0.499844	0.509213	0.503593	0.703007
1	Bagging	0.365242	0.745972	0.477684	0.802950
2	RandomForest	0.265019	0.813495	0.378919	0.808755
3	NN_MLP_Multi	0.709948	0.721602	0.715174	0.900465
4	Logistic	0.569617	0.299128	0.363526	0.640216
5	KNN_Classifier	0.308745	0.636541	0.409462	0.736597
6	SVC_Linear	0.073806	0.188034	0.093217	0.657547
7	SVC_RBF	0.276549	0.136675	0.174806	0.464860
8	Boosting_LightGBM	0.441112	0.821903	0.561057	0.833800

```
=====================================================
```

```python
# NN (Multi-output)
nn_models = {
    "NN_MLP_Multi": MultiOutputClassifier(
        MLPClassifier(
            hidden_layer_sizes=(128, 64),
            solver='lbfgs',          # lbfgs → adam
            activation='relu',
            max_iter=10000,          # 10000까지 안 가도 충분
            learning_rate_init=0.001,
            early_stopping=True,     # adam에서만 의미 있음
            random_state=seed))
    }
```

* Adam, SGD failed to learn (F1 = 0) in same condition

Separate Appendix contents

No.	Category	Contents	File Name
1	Python Code	All integrated version (dataset generation, model fitting, evaluation, etc)	Leakage_Detection_ML_Project_ Woo Jin Choi_final.py
2	Python Code	Additional Neural Network model fitting test	Leakage_Detection_ML_Project_ Woo Jin Choi_NN_test.py
3	Simulation Model	EPANET network	Net1_24hr.inp
4	Dataset	Base Demand data	Base_Demands.csv
5	Dataset	Demand Pattern data	Demand_Patterns.csv
6	Dataset	Demand (Water consumption) data	Demand.csv
7	Dataset	Final Input dataset for learning	Combined_All.csv
8	Result Summary	Result of Evaluation, Feature Importance in 1st Stage	ML_Evaluation_Results_Final.csv

* No. 4 ~ No. 8 was generated by No. 1 Python code.